CASE STUDY / 2026

Zisonix Commerce Platform

A bilingual, database-enforced commerce platform for catalogue, pricing, checkout, payments, and administration.

Next.js 16 | TypeScript | Supabase PostgreSQL | Stripe | PayPal | Resend

PREPARED BY

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

SECTION 00

Contents

A technical walkthrough of the problem, architecture, implementation, evidence, and next stage.

01	**Project Overview**	Page 3
02	**Platform Architecture**	Page 4
03	**Commerce Data Model**	Page 5
04	**The Engineering Challenge**	Page 6
05	**Implementation Approach**	Page 7
06	**Quality & Verification**	Page 8
07	**Outcome & Next Stage**	Page 9

Evidence standard

This document is based on the implemented application structure, 33 database migrations, catalogue import specification, security decision records, API routes, and executable SQL assertions. Deployment status is not inferred.

SECTION 01

Project Overview

A data-driven storefront that keeps catalogue operations, pricing logic, account access, and payment workflows consistent from database to user interface.

Zisonix Commerce turns a heterogeneous marketplace catalogue into a controlled French-first storefront with bilingual presentation, public product discovery, customer accounts, basket management, checkout, payment webhooks, order history, and an operational admin workspace.

The implementation treats commerce correctness as a data problem. Monetary values are stored as integers, tax and delivery rules are calculated centrally, product variants and bulk tiers have explicit database contracts, and Supabase row-level security protects operational data even when clients call the API directly.

210	1,311	33	159
catalogue products imported	source images registered	database migrations	SQL rule assertions

Database-first security

Every table has an explicit access policy; sensitive operational columns are never protected by UI hiding alone.

Exact money semantics

Prices use integer cents or tenths of a cent, while tax rates use basis points to eliminate floating-point invoice drift.

Import with evidence

Catalogue ingestion validates ASIN joins, prices, identifiers, stock defaults, translations, and unresolved variant groupings.

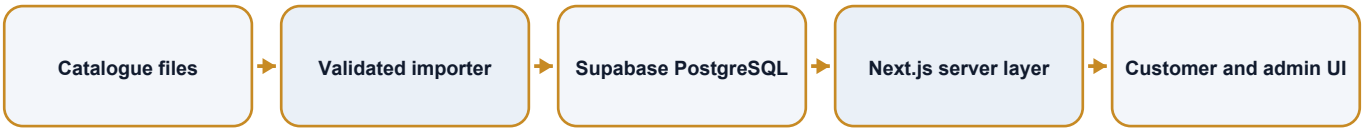
Operational completeness

The same application covers browsing, customer accounts, cart, checkout, payments, orders, imports, and administration.

SECTION 02

Platform Architecture

Next.js presentation and server routes sit above Supabase contracts that own authorization, pricing, inventory, and order integrity.



Layer	Implemented responsibility
Catalogue	CSV and image manifests provide products, variants, localized names, descriptions, pricing, stock, and media.
Data platform	PostgreSQL schemas, constraints, views, functions, and 33 migrations encode product, pricing, basket, order, delivery, and audit rules.
Security	Supabase authentication, row-level security, column grants, and server-owned order creation enforce access and prevent client-controlled totals.
Application	Next.js 16, React 19, TypeScript, next-intl, React Hook Form, Zod, and Tailwind deliver bilingual customer and staff workflows.
Payments and email	Stripe and PayPal webhook routes reconcile payment state; Resend supports transactional order communication.

Trust boundary

The browser may request a basket or checkout, but it never supplies authoritative prices, discounts, tax, delivery, or order totals. Server and database logic calculate those values from governed rules.

SECTION 03

Commerce Data Model

Products, variants, translations, prices, baskets, orders, payments, and delivery rules are modeled as separate contracts with explicit ownership.

Catalogue core

Products and variants maintain stable identifiers, categories, attributes, localized content, images, and public availability.

Pricing ladder

Unit prices resolve by quantity break, retain currency and tax context, and refuse invalid tiers that cost more than the base price.

Basket and checkout

Guest and account baskets follow the shopper, while server-side checkout recalculates every line before creating an order.

Administration

Staff workflows cover products, categories, clients, orders, pricing rules, catalogue imports, delivery, and audit visibility.

Contract	Rule	Enforcement
Money	No floating point	bigint cents / tenth-cents plus basis-point rates
Catalogue	Stable join keys	ASIN, SKU, variant identity, uniqueness constraints
Access	Least privilege	RLS, column grants, private helper schema
Orders	Server authoritative	Database pricing functions and payment reconciliation

The model is deliberately auditable: changes arrive through migrations, catalogue imports produce traceable outcomes, and business rules are asserted at the same privilege level used by real callers.

SECTION 04

The Engineering Challenge

A storefront can look correct while still leaking stock, mispricing tax, duplicating variants, or trusting manipulated checkout input.

The project combined customer experience with a demanding integrity problem: source records were incomplete, French translations covered only part of the catalogue, inventory and variant groupings required controlled defaults, and the pricing model had to survive real tax rounding at every cent value.

Incomplete localization

Only a subset of French product names arrived, so fallback behavior had to stay explicit instead of silently presenting English as translated content.

Tax rounding

Tax-free storage in whole cents cannot represent every VAT-inclusive price. Tenths of a cent preserve exact listed prices before final rounding.

Sensitive inventory

Public shoppers need availability, not exact stock counts. A coarse view exposes the right signal without revealing operational detail.

Tier boundaries

Bulk prices must change exactly at the defined quantity and never make larger orders more expensive per unit.

Variant ambiguity

Marketplace siblings can represent colors or sizes, but only verified variant groups should be merged into a single product.

Webhook integrity

Payment events can arrive more than once or out of order; reconciliation must be idempotent and tied to server-owned orders.

Engineering decision

Correctness lives below the interface. Policies, constraints, functions, triggers, and executable assertions make the security and pricing claims independently testable.

SECTION 05

Implementation Approach

The build progressed through versioned database changes, server-owned business operations, and localized user journeys.

1. Normalize catalogue

Join source manifests and commercial fields by stable identifiers; preserve unresolved gaps rather than inventing data.

2. Establish database contracts

Model catalogue, pricing, identity, baskets, orders, tax, delivery, payments, and audit behavior through migrations.

3. Enforce access

Apply RLS and column privileges to every table; centralize repeated policy predicates in private security-definer helpers.

4. Build transactional flows

Implement basket persistence, checkout quotation, order creation, payment state, stock effects, and confirmation email boundaries.

5. Deliver customer experience

Create bilingual browsing, search, product detail, authentication, account, basket, checkout, and legal pages.

6. Add staff operations

Expose controlled admin screens for catalogue, clients, orders, imports, categories, pricing, delivery, and product maintenance.

Reusable pattern

Business calculations are expressed once in PostgreSQL and reused by storefront, checkout, and tests. The same function that prices a real basket is exercised at tier boundaries by the verification suite.

- 32 application pages cover customer and administration journeys; five route handlers cover search, identity callbacks, addresses, and payment webhooks.
- Typed validation with Zod and React Hook Form reduces malformed client input before it reaches server and database contracts.
- Catalogue imports remain rerunnable and report missing translations, prices, stock, identifiers, and variant relationships explicitly.

SECTION 06

Quality & Verification

Verification targets the rules that create financial, privacy, and operational risk - not only whether pages render.

Evidence	Verified result	Why it matters
Database evolution	33 ordered migrations	Schema, policies, functions, pricing, checkout, admin, and delivery changes are reproducible.
Security assertions	159 SQL checks	Anonymous, authenticated, customer, and staff behavior is evaluated at real database privilege levels.
Catalogue evidence	210 clean ASIN joins	All supplied catalogue rows matched the manifest with no duplicate identifiers and valid prices.
Media coverage	1,311 images	The storefront can resolve product media from a tracked manifest rather than ad-hoc filenames.
Application checks	Typecheck + lint + DB verify scripts	Frontend types, code quality, and database invariants share a single verification entry point.

Current boundary

The repository proves application and database implementation. This case study does not claim a current public production deployment, completed translations for every product, or client-approved bulk pricing for every catalogue item.

- Exact stock is protected while coarse availability remains public.
- Checkout totals are recomputed server-side and cannot be supplied by a crafted browser request.
- Every French VAT-inclusive price from EUR 0.01 to EUR 1,000.00 is tested for round-trip accuracy across supported rates.

SECTION 07

Outcome & Next Stage

A commerce foundation where catalogue growth, pricing changes, and payment operations can evolve without weakening data integrity.

Safer sales

Price, tax, delivery, and stock behavior is enforced below the browser.

Faster operations

Staff manage catalogue and orders through one governed application.

Extensible model

Versioned migrations support new products, payment rules, and account workflows.

The strongest result is a connected system rather than a collection of pages. Marketplace data enters through a controlled importer, commercial rules become database contracts, customer actions use server-owned calculations, and staff operations share the same source of truth.

NEXT STAGE

- Complete high-quality French names and descriptions for the remaining catalogue.
- Replace provisional stock and bulk discount values with approved commercial data.
- Run deployment-specific payment, email, accessibility, and operational monitoring checks before launch.

LET'S TALK

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

Stack: Next.js 16 | TypeScript | Supabase PostgreSQL | Stripe | PayPal | Resend